

1. Install and Setup Environment

In this step, the required Python libraries such as **spaCy**, **NLTK**, **Pandas**, and **TextBlob** are installed and imported. The spaCy language model (en_core_web_sm) is loaded to perform advanced NLP tasks. This step prepares the system to handle text preprocessing and analysis efficiently.

```
[1]
✓ 3fs
!pip -q install spacy nltk pandas textblob
!python -m spacy download en_core_web_sm -q

12.8/12.8 MB 37.8 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in order to load all the package's dependencies. You can do this by selecting the 'Restart kernel' or 'Restart runtime' option.

[2]
✓ 1bs
import re
import pandas as pd
import nltk
from textblob import TextBlob
import spacy

nltk.download("stopwords")
nltk.download("punkt")

from nltk.corpus import stopwords

nlp = spacy.load("en_core_web_sm")

NLTK_STOP = set(stopwords.words("english"))
SPACY_STOP = nlp.Defaults.stop_words
ALL_STOP = NLTK_STOP.union(SPACY_STOP)

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
```

2. Create or Load Ticket Data

Customer support tickets are taken as input. If a dataset is not available, sample tickets are created manually. Each ticket contains unstructured text from email or chat. This step provides the raw data that will be cleaned and processed.

```
[3]
✓ 0s
tickets = [
    {"ticket_id": 1, "channel": "email", "text": "Hii team!! I cant login to my acctn since ystrdy 🙄. Reset link not wrking. Email: user123@gmail.com"},
    {"ticket_id": 2, "channel": "chat", "text": "Payment failed twice!! charged 499 INR but plan not activated. txn id: TXN88219. pls help asap."},
    {"ticket_id": 3, "channel": "email", "text": "Need invoice for last month. Company: ABC Pvt Ltd. Please send to finance@abcpvt.com"},
    {"ticket_id": 4, "channel": "chat", "text": "App keeps crashing after update v2.1.0 on iPhone 13. Shows error 504 sometimes."},
    {"ticket_id": 5, "channel": "email", "text": "I want to cancel subscription. I subscribed by mistake. My phone: +91 98765 43210"},
    {"ticket_id": 6, "channel": "chat", "text": "Unable to upload files >10mb. Getting 'timeout' error. URL: https://myapp.com/upload"},
    {"ticket_id": 7, "channel": "email", "text": "Hello, please change my email from old@mail.com to new@mail.com and update profile details."},
    {"ticket_id": 8, "channel": "chat", "text": "Server down? dashboard not loading since 10am. We are in Chennai office."}
]

df = pd.DataFrame(tickets)
df
```

	ticket_id	channel	text
0	1	email	Hii team!! I cant login to my acctn since ystr...
1	2	chat	Payment failed twice!! charged 499 INR but pla...
2	3	email	Need invoice for last month. Company: ABC Pvt ...
3	4	chat	App keeps crashing after update v2.1.0 on iPho...
4	5	email	I want to cancel subscription. I subscribed by...
5	6	chat	Unable to upload files >10mb. Getting 'timeout...
6	7	email	Hello, please change my email from old@mail.co...
7	8	chat	Server down? dashboard not loading since 10am....

Next steps: [Generate code with df](#) [New interactive sheet](#)

3. Basic Text Cleaning

- The raw ticket text usually contains noise such as URLs, email IDs, phone numbers, special symbols, and extra spaces. These unwanted elements are removed using regular expressions. Cleaning improves text quality and makes further processing more effective.
- This reduces noise in the text.

```
[4]
✓ Os
URL_RE = re.compile(r"http\S+|www\.\S+")
EMAIL_RE = re.compile(r"\b\w\.-]@(\w\.-]+\.\w+\b")
PHONE_RE = re.compile(r"\b(\+?(\d{1,3})?(\d{3,10})\b)")
WS_RE = re.compile(r"\s+")

def basic_clean(text: str) -> str:
    t = str(text).strip()
    t = URL_RE.sub(" ", t)
    t = EMAIL_RE.sub(" ", t)
    t = PHONE_RE.sub(" ", t)
    t = t.replace("\n", " ").replace("\t", " ")
    t = re.sub(r"[^A-Za-z0-9\s\.\,\!\?-\'\"]+", " ", t)
    t = WS_RE.sub(" ", t).strip()
    return t

def spell_correct_textblob(text: str) -> str:
    if len(text) > 3000:
        return text
    try:
        return str(TextBlob(text).correct())
    except:
        return text
```

4. Tokenization, Lemmatization, and Stop-word Removal

The cleaned text is processed using spaCy:

- **Tokenization** splits text into individual words.
 - **Lemmatization** converts words into their base form (e.g., *failed* → *fail*).
 - **Stop-word removal** eliminates common words like *is*, *the*, and *and* that do not add meaning.
- This step extracts only the important words from each ticket.

```
[5]
✓ Os
def spacy_preprocess(text: str):
    doc = nlp(text)
    tokens = []
    lemmas = []
    for tok in doc:
        if tok.is_space or tok.is_punct:
            continue
        low = tok.text.lower().strip()
        lem = tok.lemma_.lower().strip()
        if not low or low in ALL_STOP:
            continue
        if not lem or lem in ALL_STOP:
            continue
        tokens.append(low)
        lemmas.append(lem)
    return tokens, lemmas
```

5. Named Entity Recognition (NER)

NER identifies important real-world entities present in the text such as **person names, organizations, locations, dates, and amounts**. This converts unstructured text into structured information that can be used for analysis or automation.

```
[6] def extract_ner(text: str):  
    doc = nlp(text)  
    ents = []  
    for ent in doc.ents:  
        ents.append({  
            "start_char": ent.start_char,  
            "end_char": ent.end_char,  
            "text": ent.text,  
            "label": ent.label_  
        })  
    return ents
```

6. Apply Complete Processing Pipeline

All preprocessing steps (cleaning, tokenization, lemmatization, stop-word removal, and NER) are applied to every ticket in the dataset. The processed output includes cleaned text, important keywords, and extracted entities. This creates a structured and machine-ready dataset.

```
[7] apply_spelling = False  
  
clean_basic = []  
clean_final = []  
tokens_list = []  
lemmas_list = []  
clean_for_model = []  
ner_list = []  
  
for t in df["text"].tolist():  
    t0 = basic_clean(t)  
    t1 = spell_correct_textblob(t0) if apply_spelling else t0  
  
    toks, lems = spacy_preprocess(t1)  
    cleaned_text = " ".join(lems)  
    ents = extract_ner(t1)  
  
    clean_basic.append(t0)  
    clean_final.append(t1)  
    tokens_list.append(toks)  
    lemmas_list.append(lems)  
    clean_for_model.append(cleaned_text)  
    ner_list.append(ents)  
  
df["clean_basic"] = clean_basic  
df["clean_final"] = clean_final  
df["tokens"] = tokens_list  
df["lemmas"] = lemmas_list  
df["clean_for_model"] = clean_for_model  
df["ner_annotations"] = ner_list  
  
df[["ticket_id", "channel", "text", "clean_for_model", "ner_annotations"]]
```

	ticket_id	channel	text	clean_for_model	ner_annotations
0	1	email	Hii team!! I cant login to my acct since ystr...	hii team login acct ystrdy reset link wrke email	
1	2	chat	Payment failed twice!! charged 499 INR but pla...	payment fail twice charge 499 inr plan activat...	[{"start_char": 31, "end_char": 34, "text": "4..."}]
2	3	email	Need invoice for last month. Company: ABC Pvt ...	need invoice month company abc pvt ltd. send	[{"start_char": 17, "end_char": 27, "text": "1..."}]
3	4	chat	App keeps crashing after update v2.1.0 on iPho...	app crash update v2.1.0 iphone 13 error 504	[{"start_char": 32, "end_char": 38, "text": "V..."}]
4	5	email	I want to cancel subscription. I subscribed by...	want cancel subscription subscribe mistake phone	
5	6	chat	Unable to upload files >10mb. Getting 'timeou...	unable upload file 10 mb timeout error uri	[{"start_char": 23, "end_char": 25, "text": "1..."}]
6	7	email	Hello, please change my email from old@mail.co...	hello change email update profile detail	
7	8	chat	Server down? dashboard not loading since 10am...	server dashboard load 10 chennai office	[{"start_char": 41, "end_char": 45, "text": "1..."}]

7. Add Category Labels for Classification

A new column called **category** is added to label each ticket (e.g., Login Issue, Payment Issue, Technical Issue). These labels are required to train machine learning models for automatic ticket classification.

```
[8] ✓ Os
if "category" not in df.columns:
    df["category"] = ""

df[["ticket_id", "clean_for_model", "category"]]
```

	ticket_id	clean_for_model	category
0	1	hil team login acct ystrdy reset link wrke email	
1	2	payment fail twice charge 499 inr plan activat...	
2	3	need invoice month company abc pvt ltd. send	
3	4	app crash update v2.1.0 iphone 13 error 504	
4	5	want cancel subscription subscribe mistake phone	
5	6	unable upload file 10 mb timeout error url	
6	7	hello change email update profile detail	
7	8	server dashboard load 10 chennai office	

8. Save the Processed Dataset

The cleaned and annotated data is saved as CSV files. This allows the dataset to be reused for model training, testing, reporting, or deployment without repeating the preprocessing steps.

```
[1]
df.to_csv("tickets_cleaned_annotated.csv", index=False)
df[["ticket_id", "clean_for_model", "category"]].to_csv("tickets_for_classification.csv", index=False)

print("Saved: tickets_cleaned_annotated.csv")
print("Saved: tickets_for_classification.csv")
```

Saved: tickets_cleaned_annotated.csv
Saved: tickets_for_classification.csv

9. Verify Results (Before vs After)

The original ticket text is compared with the cleaned and processed version. This helps ensure that noise is removed correctly and meaningful information is preserved. It also validates the quality of preprocessing.

```
[1]
i = 1
print("RAW:\n", df.loc[i, "text"])
print("\nCLEAN FOR MODEL:\n", df.loc[i, "clean_for_model"])
print("\nNER:\n", df.loc[i, "ner_annotations"])
```

RAW:
Payment failed twice!! charged 499 INR but plan not activated. txn id: TXN88219. pls help asap.

CLEAN FOR MODEL:
payment fail twice charge 499 inr plan activate txn txn88219 pls help asap

NER:
[{'start_char': 31, 'end_char': 34, 'text': '499', 'label': 'CARDINAL'}, {'start_char': 35, 'end_char': 38, 'text': 'INR', 'label': 'ORG'}]